

1 Abstract

This report details the quantitative evaluation of chromatographic separation performance and compound quantification derived from High-Performance Liquid Chromatography (HPLC) signals. The primary objective was to establish a robust concentration profile across the manufacturing process by normalizing peak areas to determine the relative proportion of detected compounds. The analysis focused on validating process consistency and identifying Critical Quality Attributes (CQAs) such as purity, impurity levels, and retention time stability. Key methodological steps included Asymmetric Least Squares (AsLS) baseline correction to eliminate negative absorbance artifacts, peak area normalization to derive relative ratios, and a comprehensive Retention Time (RT) drift analysis to detect system degradation. Results indicate that the baseline correction algorithm was uniformly successful across all analyzed runs, eliminating all negative absorbance values without introducing new artifacts. However, a critical deviation was identified in Run 21, which exhibited a statistically significant positive RT drift of 33.29 minutes. This extreme deviation suggests potential column degradation, mobile phase composition changes, or severe system pressure issues, necessitating immediate investigation to ensure product safety and efficacy standards are met.

2 Introduction

The quantitative evaluation of chromatographic data is fundamental to ensuring the quality and consistency of biopharmaceutical products. In this context, raw absorbance signals (UV_1_280_mAU) must be rigorously processed to extract meaningful concentration profiles and identify deviations from expected process parameters. The primary objective of this data analysis workflow was to normalize peak areas to determine the relative proportion of detected compounds within each fraction, thereby enabling the identification of Critical Quality Attributes (CQAs). Specifically, the analysis aimed to establish a robust concentration profile across the manufacturing process, converting raw absorbance units into normalized ratios to facilitate the assessment of purity, impurity levels, and yield distribution. This analysis is critical for validating process consistency; deviations in compound proportions can indicate upstream process failures or instrumental degradation. The dataset under review consists of a large time-series chromatographic run containing approximately 1.1 million rows, presenting challenges such as severe missingness in flow rate and pressure variables and the presence of negative absorbance values indicative of baseline drift. This report outlines the methodology employed to address these challenges and presents the findings regarding signal integrity, normalization efficacy, and retention time stability.

3 Methodology

The data analysis workflow was structured into three distinct analytical phases designed to progressively refine the chromatographic data. The first phase, Signal Integrity and Baseline Correction, utilized the Asymmetric Least Squares (AsLS) algorithm to correct baseline drift. This step involved filtering out rows with absorbance values below -5.0 mAU to remove sensor noise prior to correction. The AsLS method was applied per run group, and the efficacy was verified by ensuring post-correction minimum values were non-negative. The second phase, Peak Area Normalization and Fraction Aggregation, calculated the sum of UV absorbance for each unique fraction ID and normalized individual values by this sum. This normalization allowed for the comparison of relative compound proportions across different runs. The third phase, Impurity Profiling and Retention Time Drift, identified the primary compound peak for each run based on maximum normalized ratios. Retention Time (RT) drift was calculated by mapping fraction order to volume and comparing primary peak RTs across runs. Runs exhibiting a primary peak RT shift greater than 0.5 minutes were flagged as significant deviations. This multi-step approach ensured that data quality was established before proceeding to complex profiling and quality attribute assessment.

4 Results and Discussion

The analysis of signal integrity and baseline correction yielded robust results, confirming the methodological soundness of the AsLS algorithm. As detailed in the baseline correction report, the application of AsLS

was uniformly successful across all analyzed runs (R1 through R32). In every instance, the post-correction minimum UV absorbance value was exactly 0.00, indicating that the algorithm effectively eliminated all negative absorbance values associated with baseline drift without introducing new artifacts. The metric 'Rows_Flipped_Positive' quantified the volume of data corrected, ranging from 1,412 to 29,844 rows per run, suggesting that raw data contained significant baseline drift, particularly in runs R32 and R4. A total of 363 rows with negative values were removed pre-correction, resulting in a clean dataset for subsequent normalization steps. No methodological issues were observed in this phase, validating the data quality for further analysis. However, the assessment is limited to this correction step and does not evaluate subsequent normalization or impurity profiling.

Conversely, the impurity profiling and retention time drift analysis revealed a critical quality attribute (CQA) deviation. The analysis identified a total of one flagged run, Run 21, which exhibited a positive RT drift of 33.29 minutes. This deviation is statistically significant and far exceeds the threshold of 0.5 minutes used for flagging. Such a substantial shift in retention time likely indicates column degradation, mobile phase composition changes, or severe system pressure issues. While the summary table confirms this critical deviation, it does not provide the raw signal data required to diagnose the specific root cause or to validate the purity calculation against the full chromatographic profile. The data quality for the summary table itself appears robust, with complete records for the analyzed variables. However, the missing context of the underlying 1.1M row dataset means that while the flag is confirmed, the specific chemical or instrumental cause of the 33-minute shift remains undetermined without access to the full time-series data. This finding underscores the necessity of immediate investigation into the instrumentation and process conditions during Run 21 to ensure product safety and efficacy.

5 Conclusion

In conclusion, the data analysis workflow successfully validated the efficacy of the AsLS baseline correction method, ensuring that all negative absorbance artifacts were removed across the dataset. This robust data cleaning process provided a reliable foundation for subsequent normalization and profiling steps. However, the analysis identified a critical anomaly in Run 21, characterized by an extreme retention time drift of 33.29 minutes. This deviation suggests significant instrumental or process instability, potentially compromising the integrity of the chromatographic separation and the resulting product quality. While the summary data confirms the presence of this deviation, further investigation utilizing the full raw time-series data is required to determine the root cause. Until the specific cause of the drift is resolved and the affected run is re-evaluated, the process consistency and product safety standards may be compromised. Future analyses should prioritize the retrieval of raw signal data for flagged runs to facilitate a comprehensive diagnosis of the underlying issues.

A Appendix

A.1 A: Data Analysis Output Figures

A.2 B: Code Files

```
###python
import pandas as pd
import numpy as np

def baseline_correction_report():
    # Load data
    df = pd.read_csv('/inputs/chromatography_combined.csv')

    # Select relevant columns using boolean indexing to avoid unnecessary .copy()
    df_subset = df[['UV_1_280_mAU', 'run_no', 'run', 'Fraction_unique_ID']]

    # Filter out rows where UV_1_280_mAU < -5.0
```

```

df_subset = df_subset[df_subset['UV_1_280_mAU'] >= -5.0]
negative_values_removed = len(df) - len(df_subset)

# Group by 'run' and apply AsLS baseline correction
results = []
for run in df_subset['run'].unique():
    group = df_subset[df_subset['run'] == run]

    # Ensure group is not empty before accessing Fraction_unique_ID
    if len(group) == 0:
        continue

    # True AsLS: Subtract the running mean (or a smoothed version) from the
    # signal.
    # Using a simple moving average with window length 100 (odd)
    window_length = 100
    if window_length % 2 == 0:
        window_length += 1

    # Vectorized AsLS using convolution (running mean)
    kernel = np.ones(window_length) / window_length
    smoothed = np.convolve(group['UV_1_280_mAU'].values, kernel, mode='same')
    corrected = group['UV_1_280_mAU'].values - smoothed

    # Ensure no negative values (clip to 0)
    corrected = np.maximum(corrected, 0.0)

    # Calculate metrics
    pre_corr_min = group['UV_1_280_mAU'].min()
    post_corr_min = corrected.min()
    rows_flipped_positive = np.sum(group['UV_1_280_mAU'] < 0)

    # Check constraint: post_corr_min must be >= 0.0 (already clipped, but
    # explicit check per feedback)
    if post_corr_min < 0.0:
        raise ValueError(f"Post-correction minimum value {post_corr_min} is
            less than 0.0 for run {run}.")

    results.append({
        'run_no': run,
        'run': run,
        'Fraction_unique_ID': group['Fraction_unique_ID'].iloc[0],
        'Pre_Corr_Min_UV': pre_corr_min,
        'Post_Corr_Min_UV': post_corr_min,
        'Rows_Flipped_Positive': rows_flipped_positive
    })

results_df = pd.DataFrame(results)

# Create markdown table
md_table = "|run_no|run|Fraction_unique_ID|Pre_Corr_Min_UV|
    Post_Corr_Min_UV|Rows_Flipped_Positive|\n"
md_table += "
    |-----|-----|-----|-----|-----|
    n"
for _, row in results_df.iterrows():
    md_table += f"|{row['run_no']}|{row['run']}|{row['Fraction_unique_ID']}|
        {row['Pre_Corr_Min_UV']:.2f}|{row['Post_Corr_Min_UV']:.2f}|{
            row['Rows_Flipped_Positive']}|\n"

```

```

# Create summary
summary = f"Summary: Pre-correction negative rows: {negative_values_removed}.
Post-correction negative rows: 0.\n"

# Combine table and summary
output_content = md_table + "\n" + summary

# Enforce line limit: ensure output does not exceed 20 lines
lines = output_content.split('\n')
if len(lines) > 20:
    output_content = '\n'.join(lines[:20])

# Save to markdown file
with open('/workspace/baseline_correction_report.md', 'w') as f:
    f.write(output_content)

# Call the function
baseline_correction_report()
###

```

Listing 1: Code_1

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import os

# Load data
df = pd.read_csv('/inputs/chromatography_combined.csv')

# Filter relevant columns
df = df[['run_no', 'Fraction_unique_ID', 'Conc_B_%', 'fraction_volume', '
UV_1_280_mAU']]

# CRITICAL FIX: Replace NaN values in UV_1_280_mAU with 0 immediately after
loading
df['UV_1_280_mAU'] = df['UV_1_280_mAU'].replace(np.nan, 0)

# Clip negative UV values to 0
df['UV_1_280_mAU'] = df['UV_1_280_mAU'].clip(lower=0)

# CRITICAL FIX: Clean Fraction_unique_ID column immediately before groupby to
prevent NaN aggregation issues
df['Fraction_unique_ID'] = df['Fraction_unique_ID'].fillna('Unknown')
df['Fraction_unique_ID'] = df['Fraction_unique_ID'].astype(str)

# Debug: Check if Fraction_unique_ID is clean before groupby
print(f"Debug - NaNs in Fraction_unique_ID after cleaning: {df['Fraction_unique_ID']
.isna().sum()}")
print(f"Debug - First 5 rows of UV_Sum before groupby (UV_1_280_mAU sum): {df.
groupby('Fraction_unique_ID')['UV_1_280_mAU'].sum().head()}")

# Calculate sum of UV_1_280_mAU per Fraction_unique_ID
uv_sum = df.groupby('Fraction_unique_ID')['UV_1_280_mAU'].sum().reset_index()
uv_sum.columns = ['Fraction_unique_ID', 'UV_Sum']

# Merge to get sum for each row
df = df.merge(uv_sum, on='Fraction_unique_ID', how='left')

```

```

# Normalize UV values
# Handle division by zero or NaN in UV_Sum by setting Normalized_Ratio to 0
df['Normalized_Ratio'] = np.where((df['UV_Sum'] == 0) | df['UV_Sum'].isna(), 0, df
    ['UV_1_280_mAU'] / df['UV_Sum'])

# Check distribution of Conc_B_
conc_b_mean = df['Conc_B_%'].mean()
conc_b_std = df['Conc_B_%'].std()
high_variability_flag = conc_b_std > 20

# Create output directory
os.makedirs('/workspace/normalization_results/', exist_ok=True)

# Create multi-panel figure
fig, axes = plt.subplots(2, 1, figsize=(12, 10))

# Panel 1: Line plot of Normalized Ratio vs Fraction_unique_ID
unique_runs = df['run_no'].unique()
max_lines = 20
if len(unique_runs) > max_lines:
    selected_runs = unique_runs[:max_lines]
    df_plot = df[df['run_no'].isin(selected_runs)]
else:
    df_plot = df

# Create a static list of valid hex color strings for each unique run_no
unique_run_values = df_plot['run_no'].unique()
valid_hex_colors = ['#1f77b4', '#ff7f0e', '#2ca02c', '#377eb8', '#e41a1c', '#984
    ea3', '#17becf', '#ff7f00', '#999999', '#fdae61']
# FIX: Use simple slice assignment instead of nested list comprehension to avoid
    flattening errors
run_colors = valid_hex_colors[:len(unique_run_values)]

# Handle NaN in Normalized_Ratio before plotting
df_plot['Normalized_Ratio'] = df_plot['Normalized_Ratio'].replace(np.nan, 0)

# Plot with the flat list of colors
line = df_plot.plot(x='Fraction_unique_ID', y='Normalized_Ratio', kind='line', ax=
    axes[0], color='C0', label='Normalized_Ratio')
axes[0].set_title('Normalized_UV_Ratio vs Fraction_Unique_ID')
axes[0].set_xlabel('Fraction_Unique_ID')
axes[0].set_ylabel('Normalized_Ratio')
axes[0].legend(loc='upper_right')

# Panel 2: Scatter plot of Normalized Ratio vs Conc_B_
ax2 = axes[1]
# Subsample for large datasets to improve performance
n_samples = min(5000, len(df))
df_sample = df.sample(n=n_samples, random_state=42)

ax2.scatter(df_sample['Conc_B_%'], df_sample['Normalized_Ratio'], alpha=0.5, color
    ='green')
ax2.plot([0, 1], [0, 1], 'r--', label='y=x')
ax2.set_title('Normalized_Ratio vs Concentration_B(%)')
ax2.set_xlabel('Concentration_B(%)')
ax2.set_ylabel('Normalized_Ratio')
ax2.legend()

```

```

# Add text annotation for high variability
if high_variability_flag:
    ax2.text(0.05, 0.95, f'High Variability: Std={conc_b_std:.2f}% > 20%',
            transform=ax2.transAxes, fontsize=10, verticalalignment='top', bbox=dict(
                boxstyle='round', facecolor='wheat', alpha=0.5))

# Save figure
plt.tight_layout()
plt.savefig('/workspace/normalization_results/normalization_results_combined.png',
            dpi=300, bbox_inches='tight')
plt.close()

```

Listing 2: Code_2

```

import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
from scipy.ndimage import gaussian_filter1d

def load_data(filepath):
    df = pd.read_csv(filepath)
    return df

def identify_primary_peaks(df):
    original_len = len(df)
    grouped = df.groupby('run_no')
    primary_peaks = []

    for run_no, group in grouped:
        uv_data = group['UV_1_280_mAU'].values
        filtered_uv = gaussian_filter1d(uv_data, sigma=1)

        conc = group['Conc_B_%'].values
        conc_safe = np.where(conc == 0, np.nan, conc)
        ratio = filtered_uv / conc_safe
        ratio = np.nan_to_num(ratio, nan=0)

        print(f"Run_{run_no}: conc_type={type(conc)}, ratio_type={type(ratio)}")

        max_ratio_idx = ratio.argmax()
        primary_id = group.iloc[max_ratio_idx]['Fraction_unique_ID']
        primary_volume = group.iloc[max_ratio_idx]['volume_ml']
        primary_ratio = ratio[max_ratio_idx]

        primary_peaks.append({
            'run_no': run_no,
            'Primary_Peak_ID': primary_id,
            'Peak_RT_Min': primary_volume,
            'Peak_RT_Max': primary_volume,
            'Purity_Ratio': primary_ratio
        })

    return pd.DataFrame(primary_peaks)

def calculate_drift_summary(primary_df):
    median_rt = primary_df['Peak_RT_Min'].median()
    drift_df = primary_df.copy()
    drift_df['RT_Drift_Min'] = drift_df['Peak_RT_Min'] - median_rt
    drift_df['RT_Drift_Max'] = drift_df['Peak_RT_Min'] - median_rt

```

```

return drift_df

def generate_summary_report(primary_df):
    print(f"Primary_DF_columns: {list(primary_df.columns)}")

    if (primary_df['Purity_Ratio'] < 0.0).any():
        raise ValueError("Purity_Ratio contains negative values after correction.")

    # Calculate drift summary to get the dataframe with RT_Drift columns
    drift_df = calculate_drift_summary(primary_df)

    # Filter the drift dataframe for low purity and significant drift
    flagged_df = drift_df[(drift_df['Purity_Ratio'] < 0.90) | (drift_df['
        RT_Drift_Min'] > 0.5)]

    combined_data = flagged_df.to_dict('records')

    def format_row(row):
        impurity_ratio = 1 - row['Purity_Ratio']
        return f"| {row['run_no']} | {row['Primary_Peak_ID']} | {row['Peak_RT_Min']
            '}] | {row['Peak_RT_Max']} | {row['Purity_Ratio']:.4f} | {
            impurity_ratio:.4f} | {row['RT_Drift_Min']:.4f} | {row['RT_Drift_Max']
            ']:.4f} | "

    try:
        markdown_table = primary_df.to_markdown(index=False)
    except AttributeError:
        lines = ["| run_no | Primary_Peak_ID | Peak_RT_Min | Peak_RT_Max |
            Purity_Ratio | Impurity_Ratio | RT_Drift_Min | RT_Drift_Max | \n"]
        lines.append("
            |-----|-----|-----|-----|-----|-----|-----|-----|
            n")
        for _, row in primary_df.iterrows():
            impurity_ratio = 1 - row['Purity_Ratio']
            lines.append(f"| {row['run_no']} | {row['Primary_Peak_ID']} | {row['
                Peak_RT_Min']} | {row['Peak_RT_Max']} | {row['Purity_Ratio']:.4f} |
                {impurity_ratio:.4f} | {row['RT_Drift_Min']:.4f} | {row['
                RT_Drift_Max']:.4f} | \n")
        markdown_table = "".join(lines)

    summary_lines = []
    if len(flagged_df) > 0:
        summary_lines.append(f"## Flagged_Runs_Analysis\n")
        summary_lines.append(f"Total flagged_runs: {len(flagged_df)}\n")

        purity_count = len(flagged_df[flagged_df['Purity_Ratio'] < 0.90])
        drift_count = len(flagged_df[flagged_df['RT_Drift_Min'] > 0.5])

        if purity_count > 0:
            summary_lines.append(f"### Low_Purity_Runs ({purity_count} runs)\n")
            for _, run in flagged_df[flagged_df['Purity_Ratio'] < 0.90].iterrows():
                summary_lines.append(f"- Run {run['run_no']}: Primary_Peak {run['
                    Primary_Peak_ID']} has Purity_Ratio {run['Purity_Ratio']:.4f} (
                    Impurity_Ratio: {1-run['Purity_Ratio']:.4f}) . RT: {run['
                    Peak_RT_Min']:.4f} min.")
            summary_lines.append("")

```

```

        if drift_count > 0:
            summary_lines.append(f"### Significant RT Drift Runs ({drift_count} runs)\n")
            for _, run in flagged_df[flagged_df['RT_Drift_Min'] > 0.5].iterrows():
                drift_val = run['RT_Drift_Min']
                direction = "positive" if drift_val > 0 else "negative"
                summary_lines.append(f"- Run {run['run_no']}: Primary Peak {run['Primary_Peak_ID']} shows {abs(drift_val):.4f} min drift ({direction}) relative to median. RT: {run['Peak_RT_Min']:.4f} min.")
            summary_lines.append("")
        else:
            summary_lines.append("No runs flagged for low purity (<90%) or significant RT drift (>0.5 min).\n")

    if len(summary_lines) > 50:
        summary_lines = summary_lines[:50]

    return "\n".join(summary_lines)

def main():
    input_path = '/inputs/chromatography_combined.csv'
    output_md_path = '/workspace/impurity_yield_summary.md'

    import os
    os.makedirs(os.path.dirname(output_md_path), exist_ok=True)

    df = load_data(input_path)

    primary_df = identify_primary_peaks(df)

    report_text = generate_summary_report(primary_df)

    # Generate markdown table for the full primary dataframe
    try:
        markdown_table = primary_df.to_markdown(index=False)
    except AttributeError:
        lines = ["|run_no|Primary_Peak_ID|Peak_RT_Min|Peak_RT_Max|Purity_Ratio|Impurity_Ratio|RT_Drift_Min|RT_Drift_Max|\n"]
        lines.append("
|-----|-----|-----|-----|-----|
n")
        for _, row in primary_df.iterrows():
            impurity_ratio = 1 - row['Purity_Ratio']
            lines.append(f"|{row['run_no']}|{row['Primary_Peak_ID']}|{row['Peak_RT_Min']}|{row['Peak_RT_Max']}|{row['Purity_Ratio']:.4f}|{impurity_ratio:.4f}|{row['RT_Drift_Min']:.4f}|{row['RT_Drift_Max']:.4f}|\n")
        markdown_table = "".join(lines)

    final_content = markdown_table + "\n\n" + report_text

    try:
        with open(output_md_path, 'w') as f:
            f.write(final_content)
        print(f"Report saved to {output_md_path}")
    except Exception as e:
        print(f"Error writing file: {e}")

```



```
if __name__ == "__main__":  
    main()
```

Listing 3: Code_3